HACKATHON FINAL REVIEW · EMBEDDED SYSTEMS

Smart Parking Assist

ATmega2560-based smart parking system using bare-metal drivers

> Objective: detect parking-slot occupancy with ultrasonic and IR sensors using register-level C drivers, and show the status on a seven-segment display, LEDs and a buzzer.



From Slot Sensor to Driver Display

The problem

Drivers circle the lot looking for a free slot. That wastes time and fuel, and manual checking does not scale.

Our solution

Sensors detect occupancy. Bare-metal C drivers read them and show the slot status on a display, LEDs and a buzzer.

SENSORS
HC-SR04 ultrasonic
IR sensor (ADC / digital)

→

BARE-METAL DRIVERS
gpio · timer · pwm
ultra · adc · display

→

APPLICATION LOGIC
Thresholds → parking state
FREE / OCCUPIED

→

OUTPUTS
7-segment display
LEDs · buzzer

`Arduino Mega 2560 · ATmega2560 @ 16 MHz`

CORE (per project brief): GPIO · Timer · PWM · Ultrasonic · ADC · Display. Verify each against the source.

PROPOSED: sensor filtering · non-blocking display refresh · multi-slot parking.

GPIO Driver: Register-Level Control

ILLUSTRATIVE · verify vs gpio.c

Register	Role	Bit meaning
DDRx	Direction	1 = output, 0 = input
PORTx	Output level	1 = HIGH, 0 = LOW
PINx	Input level	bit = pin state

Why a reusable driver?

- Bit-twiddling lives in one tested place
- App code reads clearly: gpio_write(LED, 1)
- Change a pin once, not in 20 places
- Reused by ultra, display, LED and buzzer

```
// ILLUSTRATIVE
void gpio_mode(volatile uint8_t *ddr,
  uint8_t pin, uint8_t out) {
  if (out) *ddr |= (1 << pin);
  else     *ddr &= ~(1 << pin);
}
uint8_t gpio_read(volatile uint8_t *pinr,
  uint8_t pin) {
  return (*pinr >> pin) & 1;
}
```

- |= (1 << pin) sets one bit, the rest stay untouched
- &= ~(1 << pin) clears one bit (the mask)
- volatile forces a real register access every time
- gpio_write() repeats the same pattern on PORTx

Timer Driver: CTC and a 1 ms Tick

EXAMPLE · confirm vs timer.c

Register	Purpose
TCCR1B	WGM12 = CTC mode · CS11, CS10 = ÷64
OCR1A	Compare value (249)
TCNT1	16-bit counter, clears at match
TIFR1	OCF1A flag set at compare match
TIMSK1	OCIE1A = interrupt enable

```
TCCR1A = 0;
TCCR1B = (1<<WGM12) | (1<<CS11) | (1<<CS10);
OCR1A  = 249;
TIMSK1 = (1<<OCIE1A);
```

If CPU = 16 MHz, prescaler = 64, OCR1A = 249

f_timer = 16 MHz / 64 = 250 kHz

1 tick = 1 / 250 kHz = 4 µs

T = (249 + 1) / 250,000 = 1 ms

Hardware timer

Counts in silicon. The CPU stays free for other work.

Software delay

_delay_ms() busy-waits and blocks everything else.

Polling: loop until OCF1A = 1, then clear it by writing 1.

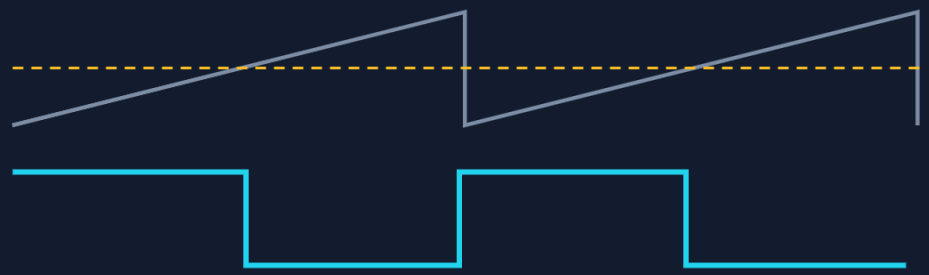
Interrupt: ISR(TIMER1_COMPA_vect) runs on every match.

Mode used in our code: [confirm in timer.c]

PWM: Fast PWM on Timer3

ILLUSTRATIVE · verify vs pwm.c

```
f_PWM = f_clk / (N × 256)
= 16,000,000 / (64 × 256)
≈ 976.56 Hz
Duty ≈ 128 / 255 ≈ 50.2 %
exact: (128 + 1) / 256 = 50.4 %
```



Grey: TCNT3 counts 0 to 255. Dashed: OCR3A. Cyan: OC3A output, HIGH until the match.

Register	Bits set	Role
TCCR3A	COM3A1, WGM30	Non-inverting · mode
TCCR3B	WGM32, CS31, CS30	Fast PWM · ÷64
DDRE	PE3	OC3A pin = output
OCR3A	128	Duty (compare value)

```
TCCR3A = (1<<COM3A1) | (1<<WGM30);
TCCR3B = (1<<WGM32) | (1<<CS31) | (1<<CS30);
DDRE |= (1<<PE3);  // OC3A = D5
OCR3A = 128;      // ~50 % duty
```

Fast PWM: one slope, f = f_clk / (N × 256) ≈ 976.56 Hz.

Phase Correct: two slopes, f = f_clk / (N × 510) ≈ 490.2 Hz.

Ultrasonic Driver: Echo Time to Distance

ILLUSTRATIVE · verify vs ultra.c

ATmega2560		HC-SR04
5 V	→	VCC
[TRIG pin] out	→	TRIG
[ECHO pin] in	←	ECHO
GND	—	GND

1. TRIG HIGH for at least 10 µs
2. Sensor sends 8 bursts at 40 kHz
3. ECHO stays HIGH for the round trip
4. Driver times ECHO, with a timeout

```
Distance = (t × v) / 2
v ≈ 343 m/s = 0.0343 cm/µs
d (cm) ≈ t (µs) / 58
e.g. t = 1160 µs → about 20 cm
```

```
// ILLUSTRATIVE
trigger_pulse();     // 10 us HIGH
t_us = echo_high_time(TIMEOUT);
return t_us / 58;    // cm, 0 = timeout
```

Common causes of a zero reading

- No echo: object outside 2–400 cm, or soft / angled
- TRIG and ECHO swapped, or 5 V / GND missing
- TRIG pulse shorter than 10 µs
- Timeout too short, or wrong timer prescaler

ADC, IR Sensor and Parking Logic

ILLUSTRATIVE · verify vs adc.c

Register	What we set
ADMUX	REFS0 = 1 (AVcc reference) · MUX = channel
ADCSRA	ADEN on · ADSC starts · ADPS = 111 (÷128, 125 kHz)
ADC	10-bit result, 0 to 1023 (ADCL / ADCH)

Aspect	Digital IR	Analog IR
Output	0 or 1	0–5 V level
Read by	gpio_read()	ADC driver
Gives	Present / absent	Level to compare

```
ADMUX  = (1<<REFS0) | (ch & 7);
ADCSRA = (1<<ADEN) | (1<<ADPS2)
       | (1<<ADPS1) | (1<<ADPS0);
ADCSRA |= (1<<ADSC);      // start
while (ADCSRA & (1<<ADSC)); // wait
return ADC;               // 0..1023
```

```
state = (reading vs [THRESHOLD])
        ? [OCCUPIED] : [FREE];
```

Use the real threshold and state names from the source only.

ADC = 512 → 512 / 1024 × 5 V = 2.5 V

A voltage level, not centimetres. IR output depends on surface and light, so it only supports threshold decisions.

Seven-Segment Display and Indicators

ILLUSTRATIVE · verify vs source



Segments a–g are shared. Digit-select lines pick one digit at a time.

Type	Common cathode	Common anode
Common pin	GND	+5 V
Segment ON	drive HIGH	drive LOW
Digit 0	0x3F	0xC0

Lookup table, common cathode, bits gfedcba

```
0:3F 1:06 2:5B 3:4F 4:66
5:6D 6:7D 7:07 8:7F 9:6F
```

Sensor to output

1. Sensor reading
2. Distance or parking state
3. Split into tens and ones
4. Lookup table gives segment pattern
5. Multiplex both digits on PORT pins
6. State drives LEDs and buzzer

Multiplexing and flicker

- Digits off, load segments, enable one digit
- Wait about 2–5 ms, then swap digits (≈100 Hz)
- Long blocking delays elsewhere cause flicker
- LED via gpio_write(); buzzer via GPIO or PWM (D5)

10 · CONCLUSION, FUTURE SCOPE AND VIVA

What We Built and Learned

- **Bare-metal drivers**
- **Direct register control**
- **Timer and PWM maths**
- **Sensor integration**
- **Modular C, .c and .h**
- **Datasheet-first debugging**

FUTURE SCOPE

- Sensor filtering (median / average)
- Non-blocking display refresh (ISR)
- Multi-slot parking, more sensors

VIVA Q&A

Why write registers directly?
Full control, smaller code, no hidden delays.

What does DDRx do?
Sets each pin: 1 = output, 0 = input.

How do we get 1 ms?
16 MHz ÷ 64 = 250 kHz; 250 ticks = 1 ms.

Is ADC 512 in centimetres?
No. It is a voltage level, about 2.5 V.